

# Extra credit 2. ngrok adapter

Not Saturday. Run the Lab 1 enhancer on your laptop. GitHub still sends real events.

New trigger. Same exits. Same harness. The graph does not change.

Answer: `solutions/extra_credit/s_ext_2_ngrok/`

No FastAPI. Stdlib `ThreadingHTTPServer`. Port 8765.

Spillwave Solutions | [spillwave.com](https://spillwave.com)

## What you will build

1. Copy the Lab 1 plugin into this folder (`task copy-plugin`)
2. Configure `config.json` and clone your CRM fork
3. Run `bin/webhook_trigger.py`
4. Give GitHub a public URL with ngrok
5. Open an issue titled `[T900] ...` and watch one `task run` start

The adapter is not the loop. It verifies, replies 202, and spawns `task run -- --ticket Txxx`.

## Why ngrok

GitHub needs a public HTTPS URL. Your laptop only has localhost.

ngrok opens an outbound tunnel. You do not open a firewall port.

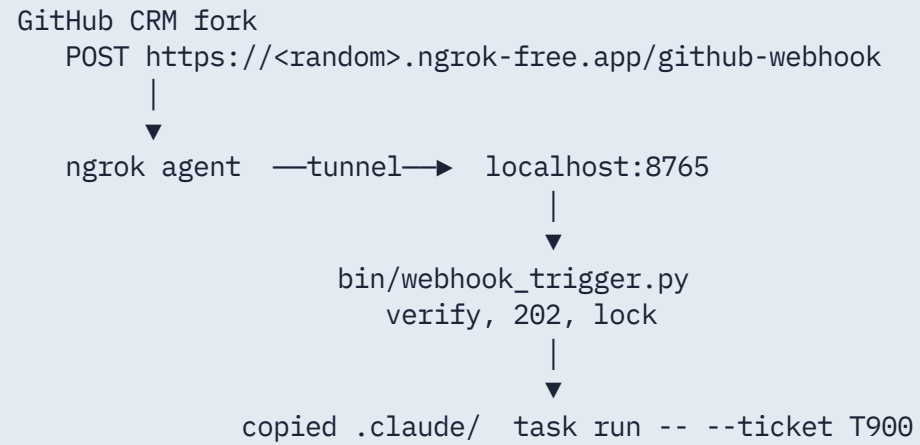
Free tier (August 2026): 3 endpoints, 1 GB, 20k HTTP requests. URLs change on restart. Paid reserved domains stay put.

## Learning objectives

- Copy the plugin without importing `sol1_enhancer` at runtime
- Listen on 8765 so the CRM on 8000 is free
- Verify HMAC in Python even if ngrok also checks at the edge
- Dedup on `X-GitHub-Delivery`
- Prove a live GitHub delivery with 202 and `work/last-webhook.json`

Spillwave Solutions | [spillwave.com](https://spillwave.com)

# Starting architecture



## Step 1. Copy the plugin here

```
cd solutions/extra_credit/s_ext_2_ngrok  
task copy-plugin
```

Copies `.claude/`, `config.json.example`, `bin/setup_test_tickets.sh`.

Does **not** copy `SPEC.md`, `HOW_TO_RUN.md`, or Lab 1's Taskfile. This folder owns those. Copied `.claude/` is gitignored.

## Step 2. Configure and prove one poll by hand

```
cp config.json.example config.json    # set fork_owner
task clone
task create-test-tickets               # optional T900 T901 T902
task run -- --ticket T001
```

That is the exact command the adapter will spawn. If this fails, ngrok will not save you.

## Step 3. Three terminals

Terminal A:

```
export GITHUB_WEBHOOK_SECRET='pick-a-long-random-string'  
task listen                # python3 bin/webhook_trigger.py
```

Terminal B:

```
ngrok http 8765
```

Copy the HTTPS URL. Webhook path is `/github-webhook`.

Terminal C:

```
curl -s http://127.0.0.1:8765/health
```

Want `"ok": true` and `"plugin_ready": true`.



## Register the webhook on the CRM fork

On **northwind-field-crm**, not this lab repo.

- Payload URL: `https://YOUR-SUBDOMAIN.ngrok-free.app/github-webhook`
- Content type: `application/json`
- Secret: same `GITHUB_WEBHOOK_SECRET`
- Events: Issues, Issue comments. Nothing else.

GitHub sends `ping`. Adapter returns `{"status": "pong"}`. Deliveries tab should show 200.

## What the adapter does. Eleven rules.

1. `GET /health` reports whether the copied plugin is present
2. Read the raw body first
3. `HMAC SHA-256 vs X-Hub-Signature-256`. Missing secret 503. Bad sig 401
4. `ping` is pong
5. Title must contain `[Txxx]`
6. `issues` opened, edited, reopened starts one poll
7. `issue_comment` created starts one poll unless the marker is in the body
8. Reply 202 before Claude starts
9. One lock file per ticket under `work/locks/`
10. One file per `X-GitHub-Delivery` under `work/deliveries/`
11. Write `work/last-webhook.json` on every accepted run

## Signature and marker. Real code.

```
def verify_signature(payload: bytes, signature: str, secret: str) -> bool:
    if not secret or not signature:
        return False
    if not signature.startswith("sha256="):
        return False
    expected = "sha256=" + hmac.new(secret.encode(), payload, hashlib.sha256).hexdigest()
    return hmac.compare_digest(expected, signature)
```

```
MARKER = "<!-- enhancer-loop -->"
if MARKER in comment:
    return None
```

## Dedup and lock

```
def already_seen(delivery_id: str) -> bool:
    path = work_dir() / "deliveries" / f"{delivery_id}.json"
    if path.exists():
        return True
    path.write_text("{} ", encoding="utf-8")
    return False

def acquire_lock(ticket: str) -> bool:
    fd = os.open(lock_path(ticket), os.O_CREAT | os.O_EXCL | os.O_WRONLY)
```

GitHub retries. At-least-once is the contract. The delivery id file is the idempotency key.

## Local HMAC smoke test

```
BODY='{ "action": "opened", "issue": { "number": 1, "title": "[T900] grey button" } }'  
SIG="sha256=$(python3 -c '...hmac...')"  
curl -s -D- -X POST http://127.0.0.1:8765/github-webhook \  
  -H "Content-Type: application/json" \  
  -H "X-GitHub-Event: issues" \  
  -H "X-GitHub-Delivery: local-test-1" \  
  -H "X-Hub-Signature-256: $SIG" \  
  --data "$BODY"
```

Unsigned posts must return 401.

## End to end

1. Open [T900] login button is grey on the CRM fork
2. GitHub posts issues/opened. Adapter replies 202
3. task run -- --ticket T900 grooms, posts a marked comment
4. You reply LGTM after the rubric is green
5. Next poll sets ready and loop: implementer

Watch: GitHub deliveries, ngrok inspector `http://127.0.0.1:4040, work/last-webhook.json, work/enhancer-T900.log.`

# Troubleshooting

| SYMPTOM                     | CAUSE                         | FIX                                                   |
|-----------------------------|-------------------------------|-------------------------------------------------------|
| Delivery is HTML            | free interstitial             | paid domain, or confirm User-Agent is GitHub-Hookshot |
| 401                         | secret mismatch               | same string in GitHub UI and env                      |
| 503 plugin not copied       | <code>.claude/</code> missing | <code>task copy-plugin</code>                         |
| 202, no Claude              | <code>task</code> not on PATH | <code>read work/enhancer-Txxx.log</code>              |
| ngrok URL 404 after restart | free URL rotated              | paste the new URL into the webhook                    |
| Loop replies forever        | missing marker filter         | adapter drops the marker                              |

## Recap

This is still a laptop demo. Close the lid and the tunnel dies.

Production is GitHub Actions, a Droplet, or Fargate. Same one-shot skill. Same state file in `.harness/`.

Spillwave Solutions | [spillwave.com](https://spillwave.com)